UNIVERSIDADE DE MOGI DAS CRUZES
ENGENHARIA DE SOFTWARE
POLÍTICAS DE SEGURANÇA DA INFORMAÇÃO

EMANUEL VICTOR
EVERSON DUARTE DE SOUZA
MAYCON WENDYL DA SILVA PEREIRA
VITOR DIAS SANTANA

RELATÓRIO TÉCNICO: IMPLEMENTAÇÃO DE ARQUITETURA SECDEVOPS COM FERRAMENTAS OPEN-SOURCE

Projeto CirculaGov

Relatório técnico apresentado à disciplina de Políticas de Segurança da Informação, sob orientação do Prof. Fabiano Bezerra Menegidio.

MOGI DAS CRUZES
2026

LISTA DE FIGURAS

Figura 1: Fluxograma da arquitetura SecDevOps do CirculaGov.....7
Figura 2: Histórico de execuções dos workflows no GitHub Actions.....14
Figura 3: Execução nº 4 reprovada pelo Trivy no PR nº 117.....15
Figura 4: Log da reprovação: cinco vulnerabilidades HIGH encontradas pelo Trivy.....16
Figura 5: Execução nº 11: status, duração e jobs concluídos com sucesso.....17
Figura 6: Execução nº 11: artefatos publicados, incluindo o SBOM.....17
Figura 7: Log da validação do SBOM no job sbom-trivy.....18
Figura 8: Log da busca de vulnerabilidades HIGH e CRITICAL na imagem.....19
Figura 9: Regra de proteção da main com os quatro checks obrigatórios.....20

LISTA DE QUADROS

Quadro 1: Workflows do repositório.....5
Quadro 2: Objeto analisado e momento de atuação de cada job.....5
Quadro 3: Artefatos publicados pelo workflow DevSecOps avançado.....6
Quadro 4: Categorias de análise de segurança utilizadas.....7
Quadro 5: Atendimento aos requisitos de segurança e qualidade.....10
Quadro 6: Condições de reprovação de cada job.....13
Quadro 7: Resultados da execução nº 11.....20
Quadro 8: Dados pessoais identificados pelo Bearer no código.....20
Quadro 9: Alertas de risco médio e baixo do ZAP e análise.....22
Quadro 10: Lacunas encontradas na auditoria e correções.....23
Quadro 11: Situação de cada comportamento esperado do pipeline.....24

SUMÁRIO

1 INTRODUÇÃO.....4
2 ARQUITETURA DA SOLUÇÃO.....5
2.1 Visão geral.....5
2.2 Relação entre os jobs.....5
2.3 Momento de atuação de cada ferramenta.....5
2.4 Armazenamento e disponibilização dos resultados.....6
2.5 Fluxograma da arquitetura.....6
3 DESCRITIVO TÉCNICO DAS FERRAMENTAS.....7
3.1 OWASP ZAP.....7
3.2 Bearer.....8
3.3 Trivy.....9
4 INTEGRAÇÃO COM OS REQUISITOS DO PROJETO.....10
5 IMPLEMENTAÇÃO E FUNCIONAMENTO.....10
5.1 Configuração geral do workflow.....10
5.2 Job privacidade-bearer.....11
5.3 Job sbom-trivy.....11
5.4 Job dast-zap.....12
5.5 Condições de sucesso ou falha.....13
6 EVIDÊNCIAS DE FUNCIONAMENTO.....13
7 RESULTADOS E ANÁLISE.....20
7.1 Visão geral.....20
7.2 Bearer.....20
7.3 Trivy.....21
7.4 OWASP ZAP.....21
7.5 Inconsistências identificadas e correções aplicadas.....23
7.6 Resultados comprovados e funcionalidades apenas configuradas.....24
7.7 Limitações.....24
8 CONSIDERAÇÕES FINAIS.....25
9 REFERÊNCIAS TÉCNICAS.....25

1 INTRODUÇÃO

Este relatório apresenta a implementação de uma arquitetura SecDevOps no projeto CirculaGov, sistema web desenvolvido em Django 5.2 com PostgreSQL 17 para apoiar prefeituras e entes públicos na gestão de livros didáticos e no tratamento de dados pessoais de alunos da rede pública. O objetivo é demonstrar, com evidências reais de execução, que controles automatizados de segurança podem ser incorporados ao ciclo de desenvolvimento utilizando exclusivamente ferramentas comunitárias e gratuitas.

A abordagem DevSecOps, também referida como SecDevOps quando se enfatiza a segurança como ponto de partida, parte da premissa de que segurança não é uma etapa isolada, executada ao final do projeto, mas uma responsabilidade contínua e compartilhada entre desenvolvimento, segurança e operações. Na prática, isso significa transformar verificações que tradicionalmente dependiam de auditorias manuais em etapas automáticas do processo de integração contínua (CI).

Na integração contínua, cada alteração enviada ao repositório dispara uma sequência de verificações. Quando ferramentas de segurança passam a fazer parte dessa sequência, uma falha de segurança passa a ser tratada como qualquer outro defeito: é detectada no momento em que é introduzida, quando o custo de correção é menor, e não semanas depois, em produção. Essa automação contribui para:

- **Redução de riscos:** vulnerabilidades em dependências, falhas de configuração e tratamento inadequado de dados pessoais são identificados antes de chegar ao ambiente de produção.
- **Rastreabilidade:** cada execução do pipeline fica registrada, vinculada a um commit específico, com seus relatórios armazenados como artefatos.
- **Confiabilidade das entregas:** o software entregue a clientes e partes interessadas passa por um conjunto padronizado e reproduzível de verificações, independentemente de quem fez a alteração.

O CirculaGov já possuía um pipeline de integração contínua no GitHub Actions (`.github/workflows/testes.yml`), voltado a testes automatizados, à validação da configuração de produção e ao teste dos limites de requisição do nginx. Este trabalho o complementa com um segundo pipeline (`.github/workflows/devsecops-avancado.yml`), que integra três ferramentas de segurança, cada uma responsável por uma classe distinta de análise: OWASP ZAP (análise dinâmica), Bearer (análise estática com foco em privacidade) e Trivy (análise de composição de software e geração de SBOM).

Os resultados apresentados nas seções 6 e 7 foram obtidos da execução nº 11 do workflow DevSecOps avancado (identificador 41855125533), disparada em 4 de novembro de 2026 pelo commit `1a52590` da branch `main`. Os relatórios produzidos pelas ferramentas foram baixados da própria execução e analisados pelos autores.

2 ARQUITETURA DA SOLUÇÃO

2.1 Visão geral

O repositório possui dois workflows independentes, ambos disparados a cada Pull Request e a cada push na branch `main`. O workflow DevSecOps avançado também pode ser disparado manualmente (`workflow_dispatch`).

Quadro 1: Workflows do repositório

Workflow	Arquivo	Jobs	Finalidade
Testes	`testes.yml`	`testes`, `nginx`	Testes automatizados, `check --deploy`, migrações pendentes e limites do nginx
DevSecOps avançado	`devsecops-avancado.yml`	`privacidade-bearer`, `sbom-trivy`, `dast-zap`	Análise estática, composição de software e análise dinâmica

Fonte: repositório do projeto, branch main, commit 1a52590.

2.2 Relação entre os jobs

Os três jobs de segurança não declaram dependência entre si (`needs`) e, por isso, executam em paralelo, cada um em sua própria máquina virtual (`ubuntu-24.04`). Essa escolha tem duas consequências:

- **Tempo:** a duração total do pipeline é a do job mais lento, e não a soma dos três. Na execução analisada, o total foi de 3 min 52 s, praticamente o tempo do job de DAST (3 min 47 s).
- **Isolamento:** a falha de uma ferramenta não impede que as outras executem e publiquem seus relatórios.

2.3 Momento de atuação de cada ferramenta

Quadro 2: Objeto analisado e momento de atuação de cada job

Job	Ferramenta	Objeto analisado	Momento
`privacidade-bearer`	Bearer	Código-fonte do repositório	Sem executar a aplicação
`sbom-trivy`	Trivy	Imagem Docker de produção, construída a partir do `Dockerfile`	Após o build da imagem

Job	Ferramenta	Objeto analisado	Momento
`dast-zap`	OWASP ZAP	Aplicação em execução (Docker Compose, Gunicorn e PostgreSQL), atrás de um proxy TLS	Com a aplicação respondendo

Fonte: arquivo devsecops-avancado.yml.

2.4 Armazenamento e disponibilização dos resultados

Cada job publica seus relatórios como artefatos da execução, por meio da action `actions/upload-artifact`. Os artefatos ficam disponíveis para download na página da execução no GitHub Actions. Todos os passos de publicação usam a condição `if: always()`, de modo que o relatório é publicado mesmo quando a ferramenta reprova a execução, que é justamente quando ele mais se faz necessário.

Quadro 3: Artefatos publicados pelo workflow DevSecOps avançado

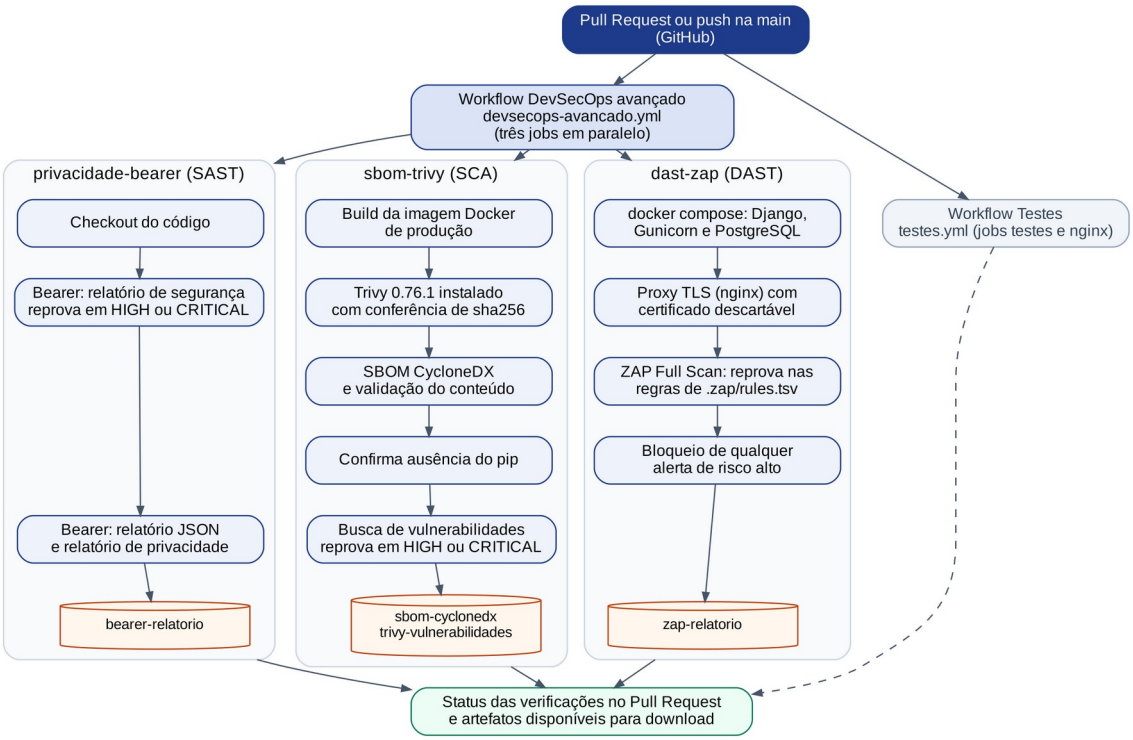
Artefato	Conteúdo
`bearer-relatorio`	Relatórios do Bearer em JSON: segurança (`relatorio.json`) e privacidade (`privacidade.json`)
`sbom-cyclonedx`	SBOM da imagem de produção, `sbom.cdx.json`
`trivy-vulnerabilidades`	Lista completa de vulnerabilidades HIGH e CRITICAL da imagem, em JSON, sem aplicar supressões
`zap-relatorio`	Relatório do ZAP em JSON, Markdown e HTML
`zap_scan`	Cópia do relatório do ZAP, publicada automaticamente pela própria action do ZAP

Fonte: arquivo devsecops-avancado.yml e página da execução nº 11.

2.5 Fluxograma da arquitetura

A Figura 1 representa o fluxo implementado, a partir do evento no GitHub até a publicação dos resultados.

Figura 1: Fluxograma da arquitetura SecDevOps do CirculaGov



Fonte: elaborado pelos autores a partir de .github/workflows/devsecops-avancado.yml.

3 DESCRITIVO TÉCNICO DAS FERRAMENTAS

As três ferramentas atuam em momentos e sobre objetos diferentes e, por isso, não se sobrepõem. O Quadro 4 resume a diferença entre as três categorias de análise.

Quadro 4: Categorias de análise de segurança utilizadas

Ferramenta	Categoria	O que analisa	Quando atua
OWASP ZAP	DAST (teste dinâmico de segurança)	A aplicação em execução, por meio de requisições HTTP	Com a aplicação rodando
Bearer	SAST (análise estática de segurança)	O código-fonte e os fluxos de dados	Sem executar a aplicação
Trivy	SCA (análise de composição de software)	Dependências e componentes de terceiros	Sobre a imagem Docker construída

Fonte: Elaborado pelos autores.

3.1 OWASP ZAP

Conceito e finalidade. O ZAP (Zed Attack Proxy) é uma ferramenta open-source de teste de segurança de aplicações web. Seu papel neste projeto é o de DAST (Dynamic Application Security Testing): avaliar a aplicação enquanto ela está em funcionamento, observando o comportamento real das respostas HTTP, sem acesso ao código-fonte.

Funcionamento técnico. O ZAP atua como um proxy entre o cliente e a aplicação. A documentação oficial descreve dois modos voltados a pipelines. O Baseline Scan executa o spider por um minuto e apenas a varredura passiva, que examina requisições e respostas sem enviar ataques. O Full Scan, utilizado neste projeto, acrescenta a varredura ativa: segundo a documentação, ele realiza ataques reais, enviando requisições maliciosas controladas, como tentativas de injeção de SQL, de comandos do sistema operacional e de scripts (XSS), e observa se a aplicação responde de forma vulnerável.

Aplicação dos testes DAST. Por realizar ataques, a varredura ativa só é adequada contra um ambiente descartável, e não contra produção. Por isso, no CirculaGov ela é executada contra uma instância criada dentro do próprio runner do GitHub Actions e destruída ao final da execução.

Importância. Certos problemas só existem em tempo de execução: um cabeçalho de segurança pode estar corretamente declarado no código e, ainda assim, ser removido por um proxy reverso ou sobrescrito por um middleware. A análise dinâmica observa o que o navegador do usuário realmente recebe e como a aplicação reage a entradas maliciosas.

Integração com o GitHub Actions. O ZAP é distribuído como imagem de contêiner e possui action oficial para o Full Scan (`zaproxy/action-full-scan`). O relatório é gerado em JSON, Markdown e HTML e publicado como artefato.

3.2 Bearer

Conceito e finalidade. O Bearer CLI é uma ferramenta de SAST (Static Application Security Testing) que analisa o código-fonte e os fluxos de dados da aplicação para identificar riscos de segurança e de privacidade. Suporta Go, Python, PHP, JavaScript, TypeScript, Ruby e Java, o que inclui a linguagem do CirculaGov.

Licença. Diferentemente do ZAP e do Trivy, distribuídos sob a licença Apache 2.0, reconhecida como open-source, o Bearer CLI é distribuído sob a Elastic License 2.0. Ele é gratuito e tem o código-fonte público, mas é classificado como source-available, porque a licença impõe restrições, como a de oferecê-lo a terceiros como serviço gerenciado. Para o uso neste projeto, essa diferença não gera custo nem limitação.

Funcionamento da análise estática. A análise é feita sobre o código, sem executar a aplicação. As regras embutidas cobrem categorias do OWASP Top 10 e do CWE Top 25, como injeção, falhas criptográficas e SSRF.

Identificação de riscos relacionados à privacidade. O diferencial do Bearer em relação a outras ferramentas de SAST é o rastreamento de fluxos de dados

sensíveis: ele identifica onde dados pessoais (PII) e dados de saúde (PHI) são manipulados e para quais componentes eles fluem, como bancos de dados, logs e APIs de terceiros. A ferramenta oferece três tipos de relatório sobre a mesma varredura: segurança, que é o padrão, privacidade e fluxo de dados.

Importância para a proteção das informações. O CirculaGov trata dados pessoais de alunos (RA, nome completo, e-mail institucional) e mantém um log de segurança que registra o nome de usuário. A análise estática com foco em privacidade permite verificar automaticamente se um dado pessoal passou a ser enviado para um destino não previsto, como um log ou um serviço externo, o que se relaciona diretamente com os princípios de minimização e finalidade adotados no projeto.

3.3 Trivy

Conceito e finalidade. O Trivy, mantido pela Aqua Security, é um scanner open-source de segurança que, neste projeto, cumpre o papel de SCA (Software Composition Analysis): analisa os componentes de terceiros dos quais o software depende.

Análise de vulnerabilidades. O Trivy cruza os componentes identificados com bases públicas de vulnerabilidades conhecidas (CVE), informando a versão afetada, a severidade e, quando existe, a versão que corrige o problema.

Inventário de dependências e geração do SBOM. Além de procurar vulnerabilidades, o Trivy gera o SBOM (Software Bill of Materials): um inventário estruturado e legível por máquina de todos os componentes do software, com nome, versão e identificador de pacote. No CirculaGov, o alvo é a imagem Docker de produção, de modo que o inventário inclui tanto as bibliotecas Python do `requirements.txt` quanto os pacotes do sistema operacional da imagem base.

Formatos de exportação. O Trivy gera SBOM nos formatos CycloneDX (JSON ou XML) e SPDX (tag-value ou JSON). O projeto usa CycloneDX, padrão mantido pela OWASP Foundation e pelo comitê técnico TC54 da Ecma International, publicado como ECMA-424. A documentação do Trivy registra um detalhe relevante: ao gerar o SBOM com `--format cyclonedx`, a busca de vulnerabilidades é desativada e precisa ser solicitada à parte, com `--scanners vuln`.

Importância da rastreabilidade. Quando uma nova vulnerabilidade é divulgada em uma biblioteca amplamente utilizada, a primeira pergunta é se o sistema usa essa biblioteca e em qual versão. Com o SBOM gerado a cada execução e armazenado como artefato, essa pergunta passa a ser respondida consultando um arquivo, sem depender de análise manual do código.

4 INTEGRAÇÃO COM OS REQUISITOS DO PROJETO

O Quadro 5 relaciona cada requisito de segurança e qualidade à ferramenta responsável e ao resultado obtido na execução analisada.

Quadro 5: Atendimento aos requisitos de segurança e qualidade

Requisito	Ferramenta	Como atende
Transparência sobre os componentes utilizados	Trivy	SBOM CycloneDX 1.7 da imagem de produção, gerado e validado a cada execução: 118 componentes na execução analisada
Identificação de dependências vulneráveis	Trivy	Busca de vulnerabilidades HIGH e CRITICAL na imagem, com reprovação do job; nenhuma vulnerabilidade corrigível encontrada na execução analisada
Validação de privacidade e proteção de dados	Bearer	Regras de segurança sobre o código e relatório de privacidade com os tipos de dado pessoal identificados
Vulnerabilidades na aplicação em execução	OWASP ZAP	Full Scan, com varredura ativa, contra a aplicação rodando em modo de produção
Rastreabilidade dos resultados	GitHub Actions	Relatórios publicados como artefatos, vinculados ao commit e à execução
Confiabilidade do software entregue	Todas	Verificações padronizadas e reproduzíveis a cada Pull Request e a cada push na `main`

Fonte: elaborado pelos autores a partir dos artefatos da execução nº 11.

Essas práticas complementam controles que o CirculaGov já possui: hash de senha com Argon2id, cifragem do segredo do 2FA com AES-GCM, integridade do log de segurança com HMAC-SHA256 encadeado e validação da configuração de produção (`check --deploy`) no pipeline de testes. As ferramentas aqui descritas não substituem esses controles; elas verificam continuamente que o restante do software, inclusive as dependências de terceiros, não introduz riscos novos.

Do ponto de vista de governança e manutenção, as verificações automatizadas reduzem a dependência de revisões manuais para riscos conhecidos e produzem registros que podem ser consultados posteriormente, o que facilita auditorias e a análise de incidentes. Este relatório não afirma conformidade legal nem certificação: as ferramentas apoiam boas práticas, mas não constituem, por si sós, comprovação de conformidade com a LGPD ou com normas técnicas.

5 IMPLEMENTAÇÃO E FUNCIONAMENTO

5.1 Configuração geral do workflow

O arquivo `.github/workflows/devsecops-avancado.yml` define:

- **Gatilhos:** `pull_request`, `push` na `main` e `workflow_dispatch`.

- **Permissões mínimas:** `contents: read`. O workflow não escreve no repositório nem abre issues (`allow_issue_writing: false` no ZAP).
- **Concorrência:** um novo push no mesmo Pull Request cancela a execução anterior, que deixa de ser relevante.
- **Fixação da cadeia de suprimentos:** todas as actions são referenciadas pelo SHA completo do commit, e as imagens de contêiner, pelo digest `sha256`. A decisão responde a um incidente real: em 19 de março de 2026, credenciais comprometidas foram usadas para reescrever 76 das 77 tags do `aquasecurity/trivy-action` com código que roubava segredos (CVE-2026-33634). Uma referência como `@v0.34.0` pode ser reapontada para outro código; um SHA de commit, não.

5.2 Job privacidade-bearer

1. Baixa o código do repositório.
2. Executa o Bearer CLI v2.1.3, em contêiner fixado por digest, com o código montado somente para leitura. A opção `--severity critical,high --exit-code 1` reprova o job se houver achado crítico ou alto.
3. Gera o relatório de segurança em JSON, com `--exit-code 0`, mesmo que o passo anterior tenha falhado.
4. Gera o relatório de privacidade em JSON (`--report privacy`), que lista os dados pessoais identificados no código e os componentes que os recebem.
5. Publica o artefato `bearer-relatorio`, com os dois relatórios.

O projeto optou pelo CLI em contêiner, e não pela action do Bearer, porque, segundo o comentário do próprio workflow, a action baixa a cada execução um script da branch principal do projeto Bearer, sem verificação de integridade.

5.3 Job sbom-trivy

1. Constrói a imagem Docker de produção (`docker build`), da mesma forma que no deploy.
2. Instala o Trivy v0.76.1 a partir do GitHub Releases, conferindo o `sha256` do arquivo antes de usá-lo. A versão é posterior às correções de dois avisos de segurança do próprio Trivy (GHSA-mcj4-mphf-j9ff e GHSA-8rc5-4fr6-64pw).
3. Gera o SBOM CycloneDX da imagem, em um arquivo JSON (`sbom.cdx.json`), com a action `aquasecurity/trivy-action`.

4. Valida o SBOM com um script Python: confere `bomFormat` igual a CycloneDX, versão de especificação 1.x e pelo menos 50 componentes, para que um SBOM vazio por erro de build não passe despercebido.
5. Publica o artefato `sbom-cyclonedx`.
6. Confirma que o `pip` não está presente na imagem, para impedir que uma alteração futura no `Dockerfile` reintroduza o problema descrito na seção 7.5.
7. Procura vulnerabilidades conhecidas de severidade HIGH e CRITICAL (`--scanners vuln`), considerando apenas as que já têm versão corrigida (`--ignore-unfixed`). São feitas duas varreduras: a primeira desliga o arquivo de supressões (`--ignorefile ""`) e gera a lista completa para o artefato; a segunda aplica o `.trivyignore`, exibe no log o que foi suprimido (`--show-suppressed`) e reprova o job se restar alguma vulnerabilidade.
8. Publica o artefato `trivy-vulnerabilidades`.

5.4 Job dast-zap

1. Gera um arquivo `.env` com chaves descartáveis, criadas na hora e destruídas com a máquina. Nenhuma chave de produção entra no pipeline.
2. Sobe a aplicação com `docker compose`, em modo de produção (`DEBUG=False`, Gunicorn e PostgreSQL).
3. Sobe um proxy TLS (nginx) com certificado autoassinado descartável, definido em `deploy/dast/nginx-dast.conf`, que repassa o cabeçalho `X-Forwarded-Proto: https`, como o nginx de produção. Sem esse proxy, com `DEBUG=False`, o Django redirecionaria toda requisição para HTTPS, os cookies `Secure`, inclusive o de CSRF, não retornariam e todo POST seria recusado, de modo que os testes de injeção não testariam nada.
4. Aguarda a aplicação responder com o código 200 em `/contas/login/`, com até 30 tentativas.
5. Executa o ZAP Full Scan 2.17.0, com imagem fixada por digest, contra `https://localhost:18443`, com spider de 3 minutos e limite de 15 minutos para a varredura ativa.
6. Lê o relatório JSON e reprova o job se houver qualquer alerta de risco alto (`riskcode` igual a 3), inclusive de regras fora da lista fixa do `.zap/rules.tsv`.
7. Exibe o log da aplicação, mesmo em caso de falha, para facilitar o diagnóstico, e publica o artefato `zap-relatorio`.

5.5 Condições de sucesso ou falha

Quadro 6: Condições de reprovação de cada job

Job	Reprova quando
`privacidade-bearer`	Há achado de severidade crítica ou alta.
`sbom-trivy`	O build falha; o Trivy baixado não confere com o `sha256` esperado; o SBOM não é CycloneDX válido ou tem menos de 50 componentes; o `pip` está na imagem; ou há vulnerabilidade HIGH ou CRITICAL com correção disponível.
`dast-zap`	A aplicação não sobe; o ZAP encontra alerta de uma das 28 regras de risco alto listadas em `.zap/rules.tsv` (`fail_action: true`); ou o relatório contém qualquer alerta de risco alto. Alertas de risco médio ou baixo aparecem no relatório, sem reprovar (opção `-I`).

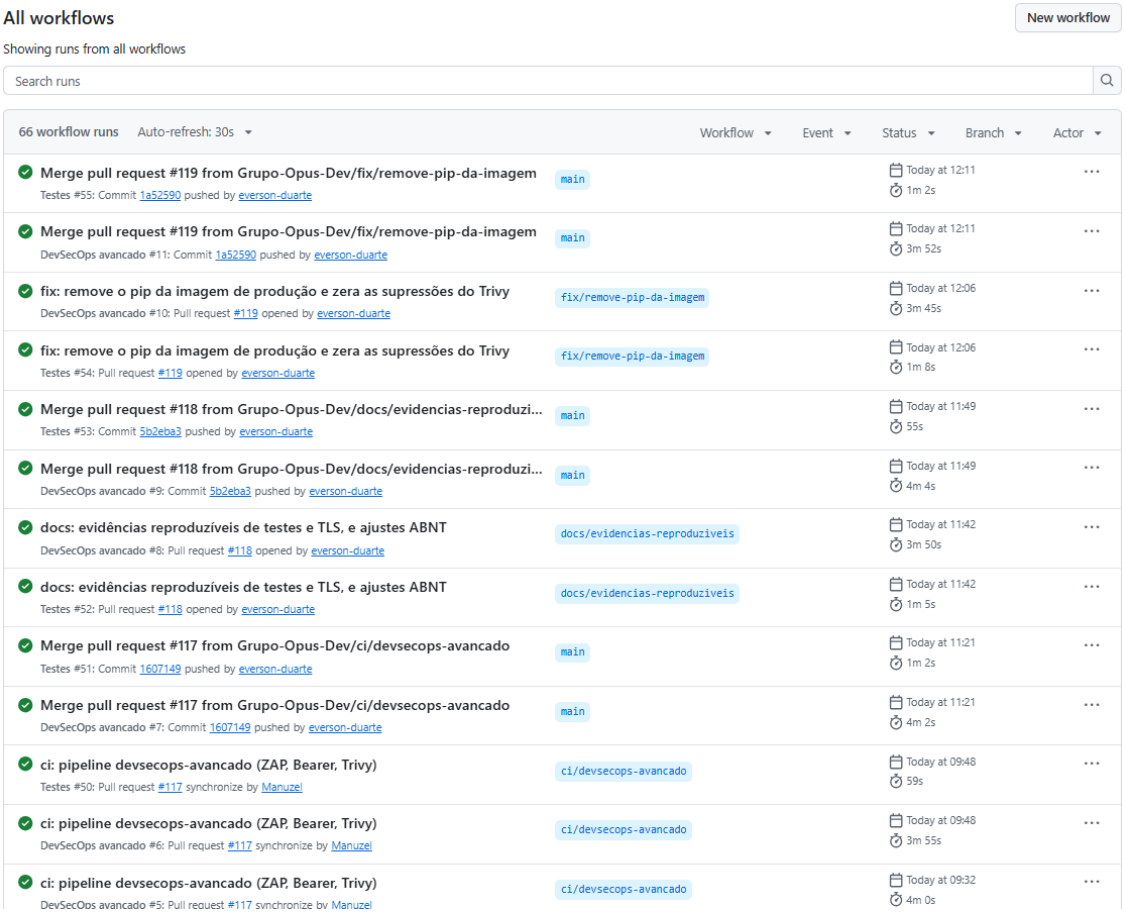
Fonte: arquivos devsecops-avancado.yml e .zap/rules.tsv.

6 EVIDÊNCIAS DE FUNCIONAMENTO

As evidências a seguir são capturas de tela reais da interface do GitHub Actions, apresentadas na ordem em que os acontecimentos ocorreram.

A Figura 2 mostra o histórico de execuções do repositório em 4 de novembro de 2026. Os dois workflows, Testes e DevSecOps avancado, são disparados juntos a cada Pull Request e a cada merge na `main`, e todas as execuções visíveis terminaram com sucesso. As duas primeiras linhas correspondem ao merge do PR nº 119, que encerrou as correções descritas na seção 7.5.

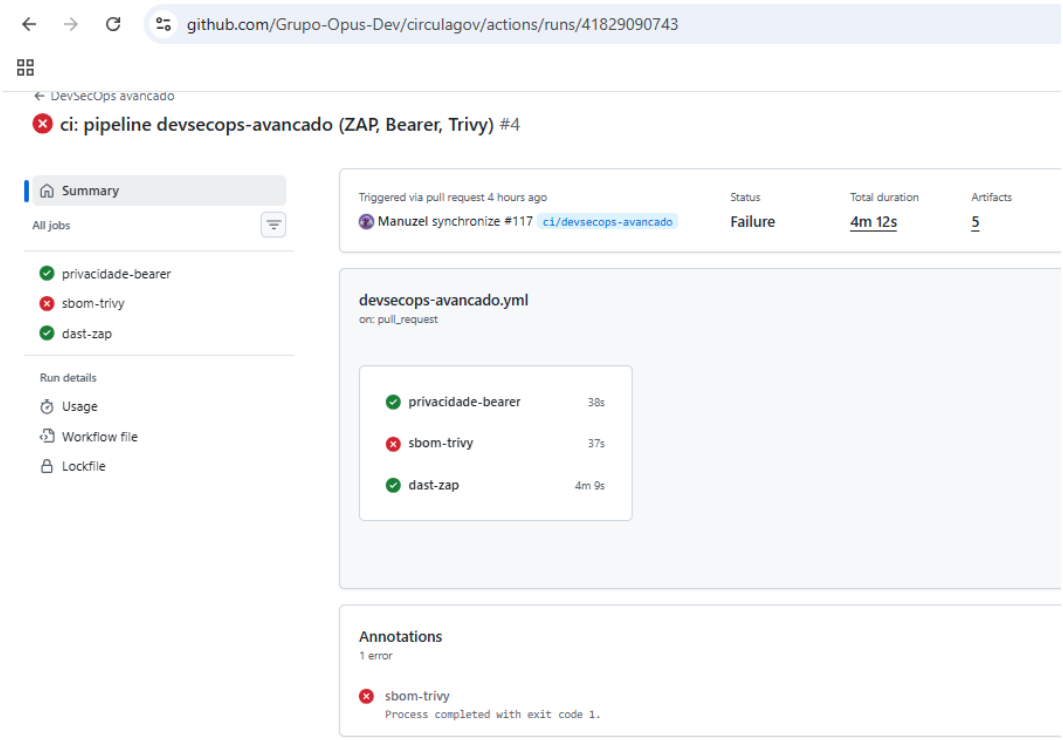
Figura 2: Histórico de execuções dos workflows no GitHub Actions



Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

Antes da execução final, o pipeline registrou uma reprovação real. A Figura 3 mostra a execução nº 4 do workflow (identificador 41829090743), disparada pelo PR nº 117 na manhã de 4 de novembro de 2026, logo após a busca de vulnerabilidades ser ativada. O status geral é Failure: os jobs `privacidade-bearer` e `dast-zap` passaram, mas o `sbom-trivy` terminou com `exit code 1`, e o Pull Request passou a exibir a verificação como reprovada.

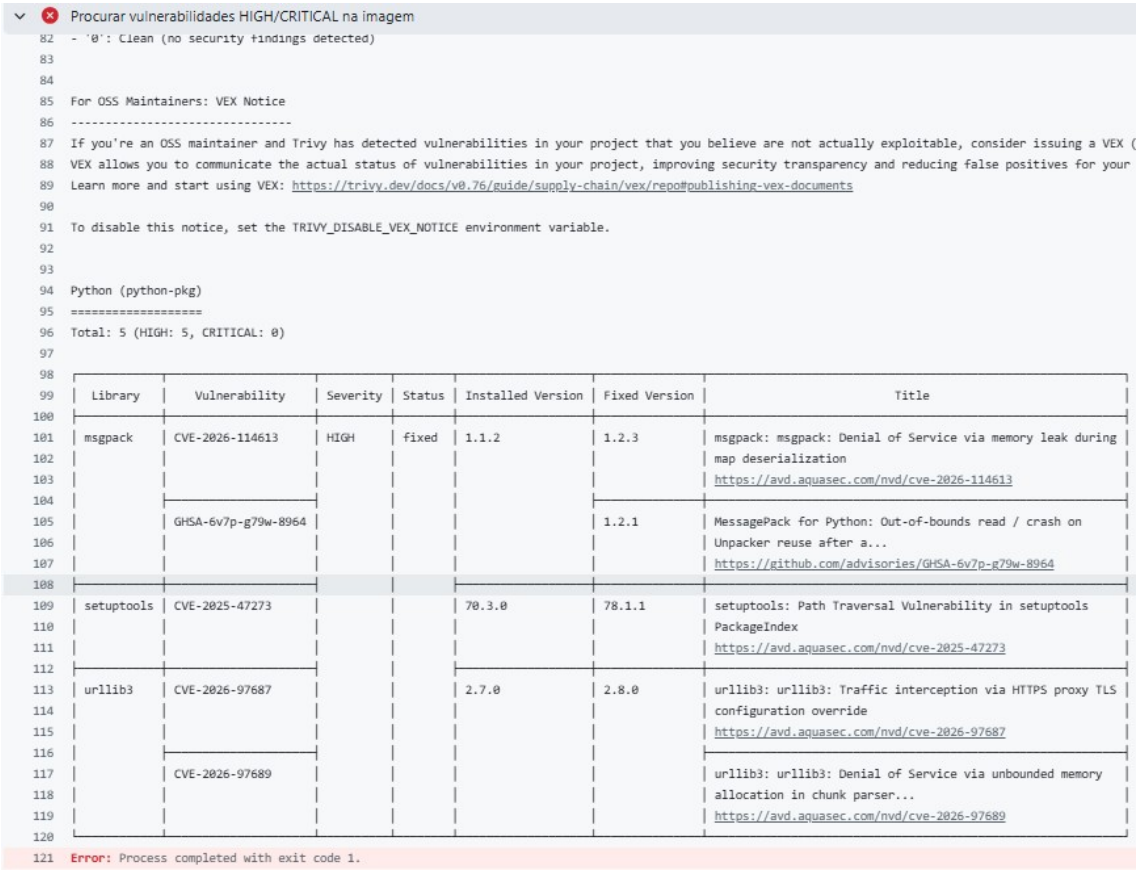
Figura 3: Execução nº 4 reprovada pelo Trivy no PR nº 117



Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

A Figura 4 mostra o motivo da reprovação, no log do passo de busca de vulnerabilidades: `Total: 5 (HIGH: 5, CRITICAL: 0)`, com as cinco vulnerabilidades, a versão instalada e a versão que as corrige. São elas CVE-2026-114613 e GHSA-6v7p-g79w-8964 (msgpack 1.1.2), CVE-2025-47273 (setuptools 70.3.0) e CVE-2026-97687 e CVE-2026-97689 (urllib3 2.7.0). O tratamento dado a elas, que levou à remoção do `pip` da imagem, é descrito na seção 7.5.

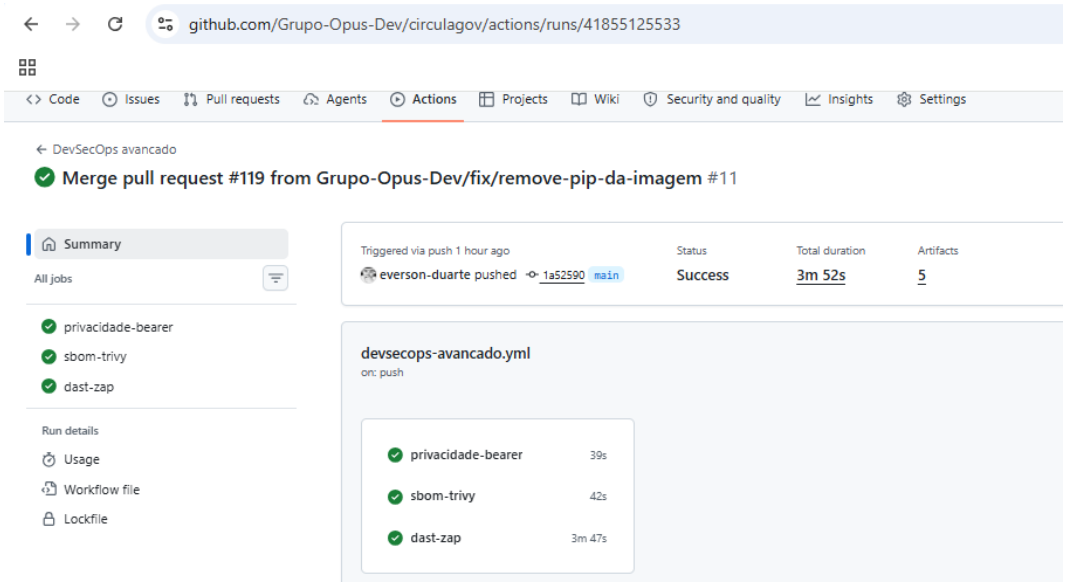
Figura 4: Log da reprovação: cinco vulnerabilidades HIGH encontradas pelo Trivy



Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

Depois das correções, a Figura 5 mostra a execução analisada neste relatório, a nº 11 do workflow DevSecOps avancado, identificada na URL pelo número 41855125533 e disparada por push na `main` (commit `1a52590`). Ela registra o status geral (Success), a duração total (3 min 52 s) e os três jobs concluídos com sucesso, com suas durações: `privacidade-bearer` em 39 s, `sbom-trivy` em 42 s e `dast-zap` em 3 min 47 s.

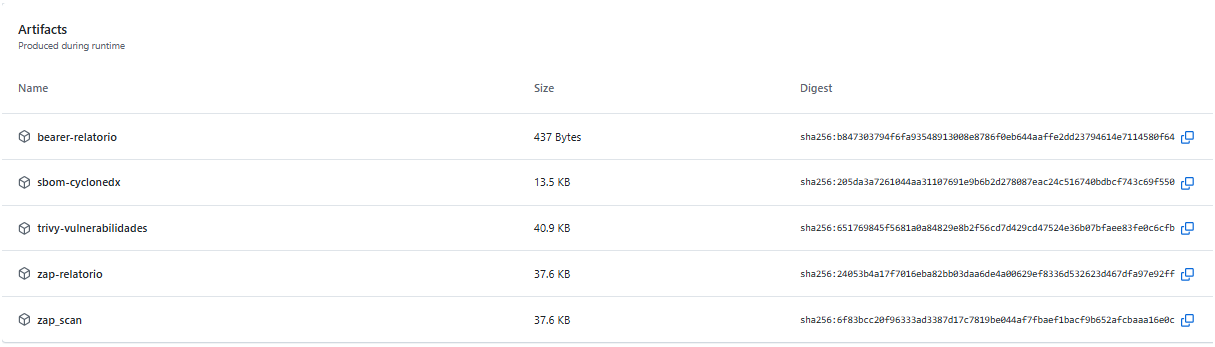
Figura 5: Execução nº 11: status, duração e jobs concluídos com sucesso



Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

A Figura 6 mostra, na mesma execução, a seção Artifacts com os cinco artefatos publicados, cada um com tamanho, digest `sha256` e botão de download. O SBOM aparece como `sbom-cyclonedx`, com 13,5 KB.

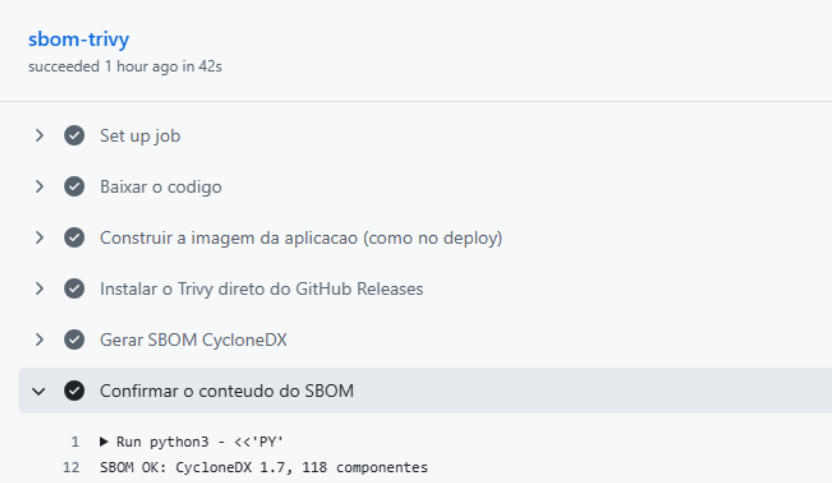
Figura 6: Execução nº 11: artefatos publicados, incluindo o SBOM



Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

A Figura 7 mostra os passos do job `sbom-trivy` na mesma execução, com o log do passo de validação aberto. A linha `SBOM OK: CycloneDX 1.7, 118 componentes` comprova que o arquivo foi gerado no formato esperado e com o inventário completo da imagem.

Figura 7: Log da validação do SBOM no job sbom-trivy



Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

Além do log, o conteúdo do SBOM foi verificado diretamente no artefato baixado. A Listagem 1 reproduz o cabeçalho do arquivo e o primeiro componente da linguagem Python, sem alteração de valores. O arquivo completo contém 118 componentes e 119 relações de dependência.

Listagem 1: Trecho do SBOM gerado na execução nº 11 (sbom.cdx.json)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:0701a8e8-1c4d-44e0-9344-9fceba9a4b70",
  "metadata": {
    "timestamp": "2026-11-04T15:11:51+00:00",
    "tools": { "components": [ { "name": "trivy", "version": "0.76.1" } ] },
    "component": { "name": "circulagov-ci:local", "type": "container" }
  },
  "components": [
    { "type": "library", "name": "Django", "version": "5.2.17",
      "purl": "pkg:pypi/django@5.2.17",
      "licenses": [ { "license": { "id": "BSD-3-Clause" } } ] },
    ... 117 componentes omitidos ...
  ]
}
```

Fonte: artefato sbom-cyclonedx da execução nº 11, trecho selecionado pelos autores.

A Figura 8 mostra o log da busca de vulnerabilidades, também no job `sbom-trivy`. A tabela lista cada alvo analisado, o sistema operacional da imagem (Debian 13.7) e cada biblioteca Python instalada, com 0 vulnerabilidades HIGH ou CRITICAL corrigíveis em todos. O `pip` não aparece na lista, o que confirma visualmente sua remoção da imagem.

Figura 8: Log da busca de vulnerabilidades HIGH e CRITICAL na imagem



sbom-trivy
succeeded 1 hour ago in 42s

Procurar vulnerabilidades HIGH/CRITICAL na imagem

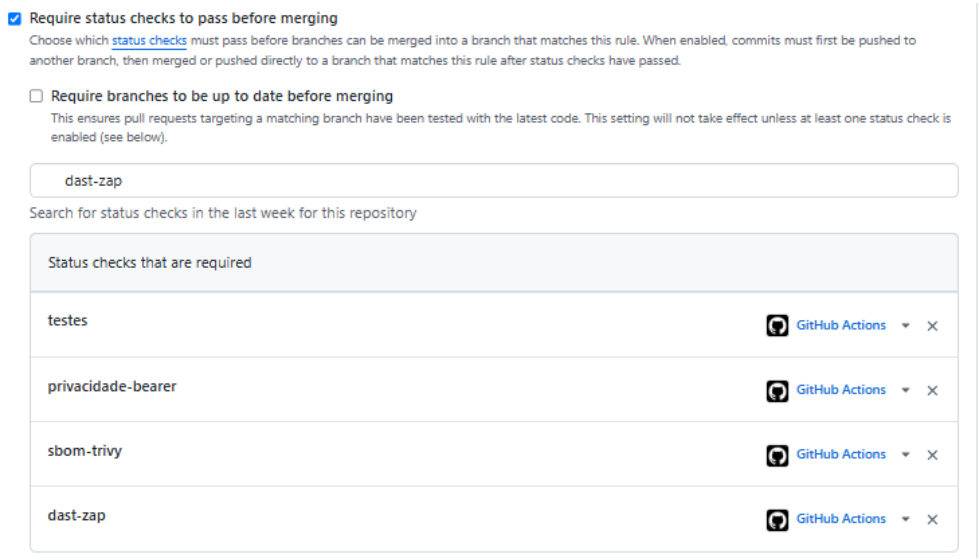
Target	Type	Vulnerabilities
circulagov-ci:local (debian 13.7)	debian	0
usr/local/lib/python3.13/site-packages/argon2_cffi-25.1.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/argon2_cffi_bindings-26.1.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/asgiref-3.12.1.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/cffi-2.1.1.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/cryptography-50.0.1.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/django-5.2.17.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/django_environ-0.14.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/django_extensions-4.1.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/gunicorn-26.2.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/markupsafe-3.0.4.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/psycopg-3.3.4.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/psycopg_binary-3.3.4.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/pycparser-3.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/pyopenssl-26.4.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/pyotp-2.10.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/qrcode-8.2.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/sqlparse-0.6.0.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/tzdata-2026.3.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/werkzeug-3.1.8.dist-info/METADATA	python-pkg	0
usr/local/lib/python3.13/site-packages/whitenoise-6.12.0.dist-info/METADATA	python-pkg	0

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

Fonte: GitHub Actions, repositório Grupo-Opus-Dev/circulagov, capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

Para que a reprovação de um job impeça de fato o avanço do código, a regra de proteção da branch `main` exige que os três jobs do workflow DevSecOps avancado, além do job `testes`, terminem com sucesso antes de qualquer merge (opção Require status checks to pass before merging). A Figura 9 mostra essa configuração. Com ela, um Pull Request reprovado pelo ZAP, pelo Bearer ou pelo Trivy não pode ser mesclado.

Figura 9: Regra de proteção da main com os quatro checks obrigatórios



Fonte: GitHub, configurações do repositório Grupo-Opus-Dev/circulagov (Branch protection rules), capturado pelos autores em 4 nov. 2026. Recorte da captura de tela.

7 RESULTADOS E ANÁLISE

7.1 Visão geral

O Quadro 7 resume o resultado de cada job na execução nº 11. Os números foram extraídos dos artefatos publicados pela própria execução.

Quadro 7: Resultados da execução nº 11

Job	Duração	Status	Resultado
`privacidade-bearer`	39 s	Sucesso	Nenhum achado de segurança, de qualquer severidade. Relatório de privacidade com 4 tipos de dado pessoal identificados, todos sem falhas.
`sbom-trivy`	42 s	Sucesso	SBOM CycloneDX 1.7 com 118 componentes. Nenhuma vulnerabilidade HIGH ou CRITICAL com correção disponível.
`dast-zap`	3 min 47 s	Sucesso	18 alertas: nenhum de risco alto, 6 de risco médio, 5 de risco baixo e 7 informativos.

Fonte: artefatos da execução nº 11, analisados pelos autores.

7.2 Bearer

O relatório de segurança (`relatorio.json`) foi gerado vazio, o que significa que nenhuma das regras de segurança do Bearer encontrou ocorrência no código, em nenhuma severidade. O relatório de privacidade (`privacidade.json`) identificou os tipos de dado pessoal apresentados no Quadro 8.

Quadro 8: Dados pessoais identificados pelo Bearer no código

Tipo de dado	Titular atribuído	Detecções	Falhas	Regras aprovadas
E-mail (Email Address)	Não identificado	10	0	40

Tipo de dado	Titular atribuído	Detecções	Falhas	Regras aprovadas
Nome de usuário (Username)	Usuário	2	0	40
Senha (Passwords)	Não identificado	1	0	40
Interações (Interactions)	Não identificado	1	0	40

Fonte: artefato bearer-relatorio da execução nº 11 do workflow DevSecOps avancado (identificador 41855125533), analisado pelos autores.

O resultado é coerente com as decisões do projeto: senha e e-mail são manipulados pelo código, mas nenhuma regra apontou que esses dados fluam para destinos indevidos, como o log de segurança, que, por decisão documentada do projeto, registra apenas o nome de usuário. Há, porém, uma limitação: o Bearer não classificou o RA e o nome completo do aluno como dados pessoais. A classificação automática se baseia em padrões de nomenclatura e não reconheceu os campos `ra` e `nome_completo`, o que reforça que o relatório de privacidade complementa, mas não substitui, o inventário manual mantido em `docs/DADOS_PESSOAIS.md`.

7.3 Trivy

O SBOM gerado está no formato CycloneDX, especificação 1.7, e inventaria 118 componentes da imagem de produção: o sistema operacional (Debian 13.7), 97 pacotes do sistema e 20 bibliotecas Python. Destas, 19 correspondem às dependências declaradas no `requirements.txt` e uma, a MarkupSafe, é dependência indireta da Werkzeug. Informação de licença está presente em 113 dos 118 componentes. O `pip` não aparece no inventário, o que confirma a correção descrita na seção 7.5.

A busca de vulnerabilidades analisou os dois grupos de componentes da imagem, pacotes do Debian e bibliotecas Python, e não encontrou nenhuma vulnerabilidade HIGH ou CRITICAL com correção disponível. Como o artefato é gerado sem aplicar o arquivo de supressões, esse resultado não decorre de vulnerabilidades ocultadas: o `.trivyignore` não suprime nenhum identificador.

7.4 OWASP ZAP

O ZAP varreu a superfície pública da aplicação, ou seja, as páginas acessíveis sem autenticação: a página inicial, que redireciona ao login, `/contas/login/`, `/recuperar-senha/`, `robots.txt`, `sitemap.xml` e os arquivos estáticos (CSS e JavaScript). Nenhum alerta de risco alto foi encontrado. O Quadro 9 apresenta os alertas de risco médio e baixo e a análise dos autores sobre cada um.

Quadro 9: Alertas de risco médio e baixo do ZAP e análise

Alerta (ID)	Risco	Ocorrência	Análise
Bypassing 403 (40038)	Médio	1 instância em `/contas/login/?next=/`	A página de login é pública e responde 200 por definição. Provável falso positivo; a regra informa confiança média.
CORS Misconfiguration (40040) e Cross-Domain Misconfiguration (10098)	Médio	`Access-Control-Allow-Origin: *` apenas nos arquivos estáticos	Comportamento padrão do WhiteNoise (opção `allow_all_origins`). O conteúdo é público e não envolve credenciais; o risco prático é baixo, mas pode ser restringido.
Insecure HTTP Method: PUT (90028)	Médio	2 instâncias em `/contas/login/`	O PUT foi recusado com 403 pela proteção CSRF; nenhuma operação foi executada. O ZAP sinaliza por esperar 405.
Proxy Disclosure (40025) e Server Leaks Version (10036)	Médio e baixo	Cabeçalho `Server: nginx/1.24.0`	Vem do proxy TLS do teste. O bloco de nginx documentado em `docs/DEPLOY.md` para produção também não define `server_tokens off`, então a recomendação vale para produção.
Sub Resource Integrity Attribute Missing (90003)	Médio	Folha de estilos do Google Fonts nas páginas	Recurso externo sem atributo `integrity`. Como o CSS do Google Fonts varia conforme o navegador, a alternativa viável é hospedar a fonte no próprio projeto.
Cookie No HttpOnly Flag (10010)	Baixo	Cookie `csrftoken`	Comportamento padrão do Django, cuja documentação considera que marcar esse cookie como HttpOnly não traz benefício prático. O cookie de sessão tem HttpOnly (`SESSION_COOKIE_HTTPONLY = True`).
Cross-Origin-Embedder-Policy, Cross-Origin-Resource-Policy e Permissions-Policy ausentes (90004, 10063)	Baixo	Respostas da aplicação	Cabeçalhos de defesa em profundidade não configurados. Podem ser acrescentados no Django ou no nginx.

Fonte: artefato zap-relatorio da execução nº 11 do workflow DevSecOps avancado (identificador 41855125533), analisado pelos autores.

Os sete alertas informativos (por exemplo, identificação da requisição de autenticação e do gerenciamento de sessão) apenas descrevem características da aplicação e não indicam vulnerabilidade.

7.5 Inconsistências identificadas e correções aplicadas

A auditoria da primeira versão do workflow (commit `4eb0ac1`) encontrou três lacunas entre o que o pipeline declarava e o que de fato fazia. Todas foram corrigidas antes da versão final (commit `16c2e0e`, mesclado no PR nº 117), conforme o Quadro 10.

Quadro 10: Lacunas encontradas na auditoria e correções

Lacuna	Evidência	Correção
O job `privacidade-bearer` não gerava relatório de privacidade	O relatório padrão do Bearer é o de segurança; o de privacidade exige `--report privacy` (documentação oficial)	Passo novo com `--report privacy`
O job `sbom-trivy` não procurava vulnerabilidades	`--format cyclonedx` desativa a busca de vulnerabilidades, que exige `--scanners vuln` (documentação oficial do Trivy)	Passo novo com `--scanners vuln`, que reprova em HIGH ou CRITICAL corrigível
O `.zap/rules.tsv` citava o passo "Bloquear alertas de risco alto", inexistente	Busca no arquivo do workflow	Passo criado

Fonte: histórico de commits do repositório.

Ao ativar a busca de vulnerabilidades, o Trivy reprovou o job com cinco vulnerabilidades de severidade HIGH: CVE-2025-47273 (setuptools), CVE-2026-97687 e CVE-2026-97689 (urllib3), CVE-2026-114613 e GHSA-6v7p-g79w-8964 (msgpack). Nenhum desses pacotes faz parte do `requirements.txt`: são cópias embutidas no próprio `pip`, no diretório `pip/_vendor`, presentes na imagem base do Python. O tratamento passou por três etapas, todas registradas no histórico do repositório:

1. **Supressão justificada** (commit `0eecc5c`): as cinco foram listadas no `.trivyignore`, com a justificativa de que o `pip` não é executado em produção.
2. **Correção da evidência** (commit `a01fa34`): verificou-se que o Trivy aplica o `.trivyignore` por padrão, de modo que as vulnerabilidades suprimidas também desapareciam do artefato, o que o tornava inútil como evidência. O primeiro scan passou a usar `--ignorefile ""` e o segundo, `--show-suppressed`.
3. **Correção definitiva** (commit `e60a594`, PR nº 119): em vez de aceitar o risco, o `pip` foi removido da imagem de produção, no mesmo comando que instala as dependências, já que nada em produção o utiliza. As vulnerabilidades deixaram de existir na imagem, o `.trivyignore` ficou sem nenhuma supressão e um passo novo do pipeline impede que o `pip` volte sem ser percebido.

Esse percurso ilustra o objetivo do pipeline: a ferramenta identificou um risco real, o risco foi tratado na origem e a regra de reprovação não foi afrouxada para obter um resultado verde.

7.6 Resultados comprovados e funcionalidades apenas configuradas

Quadro 11: Situação de cada comportamento esperado do pipeline

Comportamento	Situação	Base
Os três jobs executam com sucesso	Comprovado	Execução nº 11 (Figura 5)
Artefatos disponíveis para download	Comprovado	Execução nº 11 (Figura 6)
SBOM gerado e disponível para download	Comprovado	Artefato baixado e analisado: CycloneDX 1.7, 118 componentes
Busca de vulnerabilidades na imagem	Comprovado	Artefato `trivy-vulnerabilidades` e log do job (Figura 8)
Relatório de privacidade	Comprovado	Artefato `bearer-relatorio`
Reprovação por vulnerabilidade HIGH	Comprovado	Execução nº 4, PR nº 117 (Figuras 3 e 4)
Reprovação do ZAP por alerta de risco alto	Configurado	Não exercitado: nenhum alerta alto ocorreu
Reprovação do Bearer por achado crítico ou alto	Configurado	Não exercitado: nenhum achado ocorreu
Bloqueio de merge na `main` por falha dos jobs DevSecOps	Configurado	Os quatro checks são obrigatórios na regra de proteção (Figura 9); ainda não houve PR bloqueado após a configuração

Fonte: Elaborado pelos autores.

7.7 Limitações

- **Cobertura do DAST:** o ZAP testou apenas a superfície não autenticada. As telas que exigem login, como consulta e exclusão de dados pessoais, consentimentos e auditoria, não foram cobertas. Uma varredura autenticada é a principal evolução possível.
- **Vulnerabilidades sem correção:** a opção `--ignore-unfixed` exclui vulnerabilidades ainda sem versão corrigida, que não reprovam o job e não aparecem no artefato. Segundo a mensagem do commit `0eecc5c`, a imagem base Debian 13.7 apresentava 45 avisos HIGH nessa condição. Elas exigem acompanhamento periódico ou troca por uma imagem base menor.
- **Classificação automática de dados pessoais:** como visto na seção 7.2, o Bearer não reconheceu o RA e o nome completo do aluno.
- **Ambiente de teste:** o DAST roda atrás de um proxy TLS próprio do pipeline, e não do nginx de produção; alertas sobre cabeçalhos do servidor refletem esse

proxy, embora a análise do Quadro anterior mostre que um deles também se aplica à configuração de produção documentada.

8 CONSIDERAÇÕES FINAIS

A implementação demonstrou que é possível incorporar ao ciclo de desenvolvimento do CirculaGov três camadas complementares de verificação de segurança, análise estática com foco em privacidade, análise de composição com geração de SBOM e análise dinâmica da aplicação em execução, utilizando exclusivamente ferramentas open-source e gratuitas, integradas ao GitHub Actions e executadas a cada Pull Request.

Mais do que a execução bem-sucedida, os resultados mostram o pipeline cumprindo sua função: a auditoria identificou lacunas entre o que o workflow declarava e o que fazia, a busca de vulnerabilidades reprovou uma alteração ao encontrar cinco vulnerabilidades reais e a equipe tratou o problema na origem, removendo o componente desnecessário da imagem, em vez de afrouxar a regra. Cada etapa ficou registrada no histórico do repositório e nos artefatos das execuções, o que torna o processo rastreável e reproduzível.

Como próximos passos, recomenda-se: configurar a varredura autenticada do ZAP; definir `server_tokens off` no nginx de produção; hospedar localmente a fonte hoje carregada do Google Fonts; acrescentar os cabeçalhos Cross-Origin-Embedder-Policy, Cross-Origin-Resource-Policy e Permissions-Policy; e acompanhar as vulnerabilidades sem correção da imagem base. Com essas medidas, a solução contribui para a entrega de um software mais seguro, rastreável e confiável aos usuários e às partes interessadas.

9 REFERÊNCIAS TÉCNICAS

AQUA SECURITY. **Trivy documentation**: SBOM. [S. l.], 2026. Disponível em: https://trivy.dev/latest/docs/supply-chain/sbom. Acesso em: 4 nov. 2026.

BEARER. **Bearer CLI documentation**. [S. l.], 2026. Disponível em: https://docs.bearer.com/. Acesso em: 4 nov. 2026.

BEARER. **Report types**. [S. l.], 2026. Disponível em: https://docs.bearer.com/explanations/reports/. Acesso em: 4 nov. 2026.

CIRCL. **CVE-2026-33634**: Aquasecurity Trivy supply chain compromise. Luxemburgo, 2026. Disponível em: https://cve.circl.lu/cve/CVE-2026-33634. Acesso em: 4 nov. 2026.

DJANGO SOFTWARE FOUNDATION. **Ticket #27611**: remove check that suggests enabling CSRF_COOKIE_HTTPONLY setting. [S. l.], 2016. Disponível em: https://code.djangoproject.com/ticket/27611. Acesso em: 4 nov. 2026.

GITHUB. **Workflow artifacts**. GitHub Docs. [S. l.], 2026. Disponível em: https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts. Acesso em: 4 nov. 2026.

OWASP FOUNDATION. **OWASP CycloneDX (ECMA-424)**. [S. l.], 2026. Disponível em: https://owasp.org/www-project-cyclonedx/. Acesso em: 4 nov. 2026.

WHITENOISE. **Using WhiteNoise with any WSGI application**. [S. l.], 2026. Disponível em: https://whitenoise.readthedocs.io/en/stable/base.html. Acesso em: 4 nov. 2026.

ZAP. **ZAP: baseline scan**. [S. l.], 2026. Disponível em: https://www.zaproxy.org/docs/docker/baseline-scan/. Acesso em: 4 nov. 2026.

ZAP. **ZAP: full scan**. [S. l.], 2026. Disponível em: https://www.zaproxy.org/docs/docker/full-scan/. Acesso em: 4 nov. 2026.

ZAP. **ZAP docker user guide**. [S. l.], 2026. Disponível em: https://www.zaproxy.org/docs/docker/about/. Acesso em: 4 nov. 2026.